

MEMORIA TÉCNICA DEL PROYECTO TFG LOOPSKILL

Pierangelo Guerrero // Alejandro Jiménez Ruiz

DAW II ; IES LOPE DE VEGA

Contenido

1. Introducción.....	2
1.1 Objetivo.....	2
1.2 Descripción general del proyecto.....	2
1.3 Licencia del proyecto	3
2. Tecnologías empleadas.....	3
2.1 Angular standalone	3
2.2 TypeScript, HTML y SCSS.....	4
2.3 Servicios frontend y consumo de API REST.....	4
2.4 Node.js y Express como backend.....	5
2.5 MySQL / MariaDB.....	5
2.6 GitHub, Figma y AWS	5
3. Desarrollo del proyecto: diseño y estructura	6
4. Desarrollo del proyecto: funcionalidades y etapas	12
5. Base de datos: diseño y estructura	14
6. Pruebas, errores cometidos y soluciones planteadas	15
7. Conclusiones.....	17
8. Bibliografía	19

1. Introducción

1.1 Objetivo

El objetivo de esta memoria técnica es documentar el estado actualizado del proyecto LoopSkill, una plataforma web de formación online concebida como un entorno MOOC con recomendaciones personalizadas, control de acceso por planes y seguimiento del progreso del alumnado.

La primera versión de la memoria se centraba en el frontend funcional en modo mock. En el estado actual del proyecto, esa base ha evolucionado hacia una aplicación más integrada: el frontend Angular ya consume una API REST desarrollada con Node.js y Express para las partes principales del sistema, como autenticación, catálogo de cursos, detalle de curso, lecciones, recomendaciones, inscripciones y progreso.

La memoria mantiene la misma estructura general del documento inicial, pero actualiza el contenido para reflejar con mayor precisión el alcance real del repositorio. Por tanto, no se presenta LoopSkill como un producto completamente terminado, sino como una versión avanzada del TFG en la que ya existe una arquitectura cliente-servidor funcional, una base de datos relacional poblada y una integración parcial pero significativa entre frontend y backend.

1.2 Descripción general del proyecto

LoopSkill nace con la idea de ofrecer una experiencia de aprendizaje online más clara, ligera y personalizada que la de muchas plataformas generalistas. El proyecto mantiene la arquitectura definida desde el anteproyecto: Angular para el frontend, Node.js con Express para el backend, MySQL/MariaDB como sistema de base de datos y AWS como entorno previsto para el despliegue final.

La aplicación permite al usuario registrarse, iniciar sesión, seleccionar intereses, explorar un catálogo de cursos técnicos, consultar detalles completos de cada curso, acceder a lecciones, inscribirse, actualizar su progreso y consultar sus cursos en marcha o completados. También incluye una sección de planes Free, Pro y Premium, ajustes de cuenta e intereses, y un panel de administración accesible para usuarios con rol admin.

La principal diferencia respecto a la memoria anterior es que el backend ya no es solo una previsión arquitectónica. Actualmente existe una API REST con rutas para autenticación, usuario autenticado, etiquetas de interés, catálogo de cursos, cursos populares, recomendaciones, detalle de curso, lecciones e inscripciones. El frontend utiliza HttpClient y tokens JWT para comunicarse con esa API. Aun así, algunas partes siguen funcionando en modo mock o local, especialmente la actualización de planes y el CRUD administrativo de cursos.

1.3 Licencia del proyecto

El proyecto se distribuye bajo licencia MIT, una licencia de software libre y permisiva que permite reutilizar, modificar y distribuir el código manteniendo la referencia a la autoría original. Esta elección resulta adecuada para un proyecto académico con posible continuidad posterior, ya que facilita que el código pueda evolucionar sin introducir restricciones innecesarias.

2. Tecnologías empleadas

2.1 Angular standalone

El frontend se ha construido con Angular utilizando el enfoque standalone. Esta decisión reduce la complejidad inicial de la aplicación, evita una dependencia excesiva de módulos y favorece una organización por funcionalidades. Cada pantalla principal del sistema, como Home, Explore, Course Detail, Course Player, Plans, Settings o Auth, se implementa como un componente independiente.

Angular aporta routing, data binding, formularios, directivas estructurales y una separación clara entre vista, lógica y servicios. En el estado actual del proyecto estas capacidades se utilizan para proteger rutas mediante authGuard, mostrar contenido según el estado del usuario, gestionar formularios de cuenta y contraseña, y cambiar la experiencia de navegación según si el usuario tiene cursos inscritos o no.

2.2 TypeScript, HTML y SCSS

TypeScript se utiliza como lenguaje principal de la lógica del frontend. Su sistema de tipado permite mantener coherencia en modelos como User, Course, Lesson, Plan, Enrollment o las respuestas genéricas de la API. Esta capa tipada ha sido importante para pasar de un frontend basado en mocks a un frontend que consume datos reales desde Express.

HTML se emplea en las plantillas de cada componente con un enfoque declarativo. SCSS se utiliza para mantener una identidad visual consistente en toda la aplicación: fondo claro, cards blancas, bordes suaves, jerarquía visual limpia y verde de LoopSkill como color principal. Componentes como Course Detail, Course Player, Plans, My Learning y Settings comparten una misma lógica visual, lo que refuerza la sensación de producto coherente.

2.3 Servicios frontend y consumo de API REST

En la versión inicial, los servicios frontend utilizaban principalmente LocalStorage como mecanismo de persistencia mock. En el estado actual, los servicios principales ya consumen endpoints reales del backend mediante HttpClient.

AuthService gestiona registro, login, sesión, usuario autenticado, actualización de perfil, cambio de contraseña, actualización de intereses y carga de etiquetas disponibles. Para ello utiliza la API `/api/auth` y guarda localmente tanto el usuario actual como el token JWT.

CourseService consulta el catálogo desde `/api/courses`, permite aplicar filtros por categoría, nivel, etiquetas y búsqueda, carga cursos populares, recomendaciones personalizadas, detalle de curso y lecciones. Aún conserva una parte local para la administración mock de cursos.

EnrollmentService consume la API `/api/enrollments` para crear inscripciones, obtener cursos del usuario, separar cursos en progreso y completados, y actualizar el porcentaje de avance. También mantiene funciones mock heredadas para compatibilidad con partes antiguas del frontend.

Esta estrategia demuestra una evolución clara: la lógica importante se ha ido desplazando desde mocks locales hacia una API real, manteniendo los servicios como punto de adaptación entre interfaz y backend.

2.4 Node.js y Express como backend

El backend está implementado con Node.js y Express. La aplicación se organiza en `app.js`, `server.js`, rutas, controladores, middleware, utilidades de validación, utilidades HTTP y configuración de base de datos. La API expone un endpoint de salud (GET `/api/health`) y agrupa la lógica funcional en tres bloques principales: autenticación, cursos e inscripciones.

La autenticación utiliza `bcrypt` para el hash de contraseñas y `jsonwebtoken` para emitir y validar tokens JWT. El middleware de autenticación permite proteger rutas y comprobar roles, incluido el acceso de administración. También se han incorporado validaciones de entrada para email, contraseña, nombre, intereses y progreso.

El backend actual ya permite registrar usuarios, iniciar sesión, recuperar el usuario autenticado, actualizar perfil, cambiar contraseña, actualizar intereses, listar etiquetas, consultar cursos, obtener cursos populares y recomendados, consultar detalle de curso con outcomes y lessons, crear inscripciones y actualizar progreso. La configuración de CORS permite desarrollo local abierto y restricción por `FRONTEND_ORIGIN` en despliegue.

2.5 MySQL / MariaDB

La base de datos relacional del proyecto se ha modelado en MySQL/MariaDB y se encuentra definida en `database/mooc_db_v4.sql`. El esquema contiene tablas para categorías, planes, características de planes, etiquetas, usuarios, cursos, resultados de aprendizaje, lecciones, relaciones curso-etiqueta, intereses de usuario e inscripciones.

La conexión se realiza desde el backend mediante `mysql2/promise` y un pool de conexiones configurado con variables de entorno. El fichero `config/db.js` permite activar SSL mediante variables específicas, lo que deja preparada la aplicación para una base de datos alojada en RDS u otro entorno remoto.

2.6 GitHub, Figma y AWS

GitHub se utiliza como repositorio principal y soporte de versionado. Figma se empleó en las fases iniciales para definir la experiencia visual general y las pantallas principales. AWS sigue siendo la plataforma prevista para el despliegue final.

El repositorio ya incluye una guía de despliegue en AWS que propone S3 y CloudFront para el frontend, Elastic Beanstalk para el backend y RDS para MySQL/MariaDB. También existe un paquete backend-elastic-beanstalk.zip, lo que indica una preparación específica para desplegar la parte servidor.

3. Desarrollo del proyecto: diseño y estructura

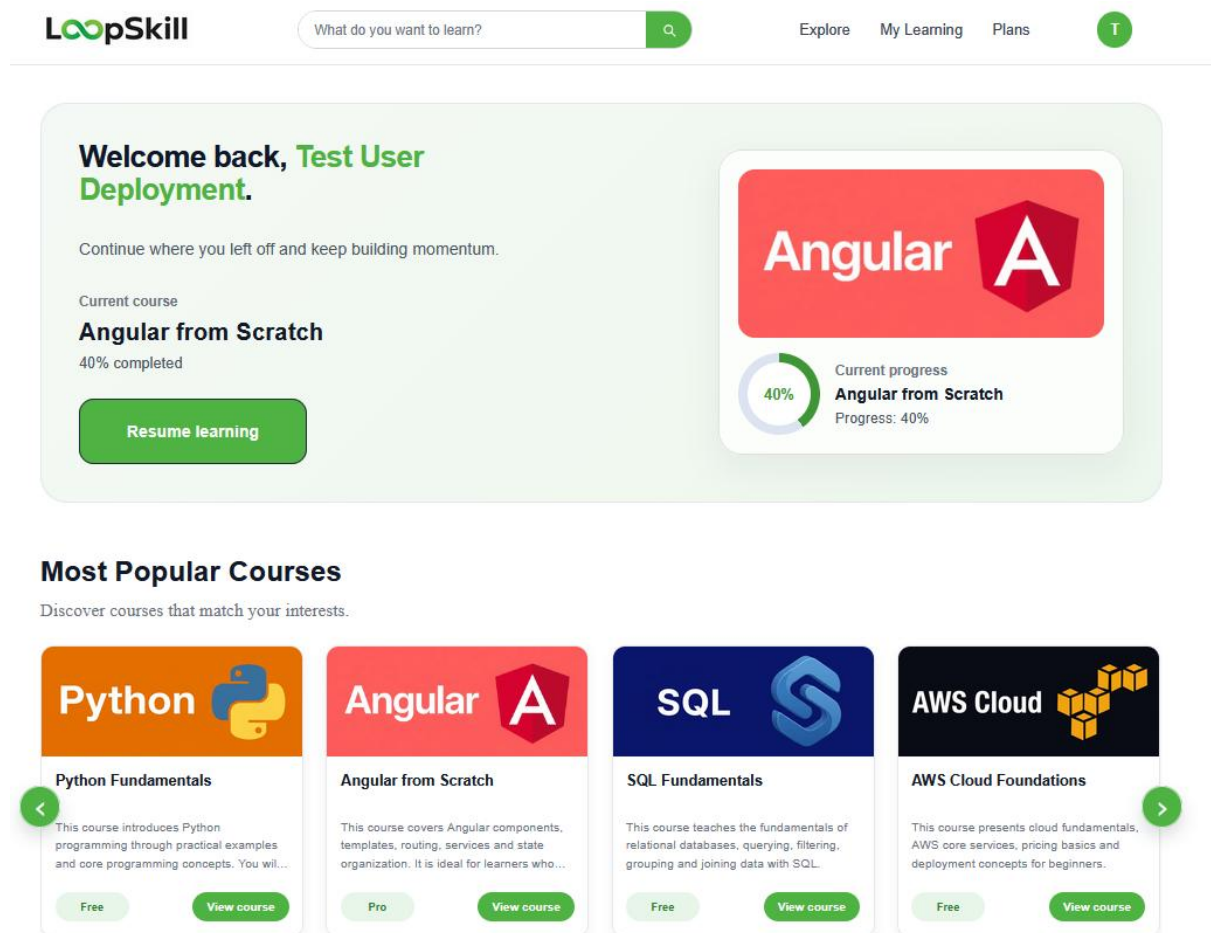
La estructura del proyecto se organiza en tres bloques principales: frontend, backend y database. Esta separación responde a la arquitectura cliente-servidor planteada desde el inicio.

```
frontend/  
├── app/  
│   ├── angular.json  
│   ├── package.json  
│   ├── tsconfig*.json  
│   └── src/  
│       ├── index.html  
│       ├── main.ts  
│       ├── styles.scss  
│       ├── environments/  
│       │   ├── environment.ts  
│       │   └── environment.prod.ts  
│       ├── assets/  
│       │   ├── images/  
│       │   │   ├── categories/  
│       │   │   └── courses/  
│       │   └── videos/  
│       └── app/  
│           ├── app.component.*  
│           ├── app.config.ts  
│           ├── app.routes.ts  
│           ├── core/  
│           │   ├── guards/  
│           │   ├── layout/  
│           │   ├── mocks/  
│           │   ├── models/  
│           │   └── services/  
│           ├── features/  
│           │   ├── auth/  
│           │   ├── course-detail/  
│           │   ├── course-player/  
│           │   ├── explore/  
│           │   ├── home/  
│           │   ├── onboarding/  
│           │   ├── plans/  
│           │   ├── settings/  
│           │   └── user-page/  
│           └── shared/  
│               ├── components/  
│               │   ├── category-card/  
│               │   └── course-card/
```

En el frontend, la carpeta src/app se divide en core, shared y features. core contiene servicios, modelos, guards, mocks y layout común. shared agrupa componentes reutilizables como tarjetas de cursos y categorías. features contiene las pantallas principales: autenticación, onboarding, home, explore, detalle de curso, reproductor de curso, planes, ajustes y página de aprendizaje.

La aplicación protege las rutas principales mediante authGuard. El flujo inicial redirige al usuario a autenticación, después puede pasar por onboarding de intereses y finalmente acceder al catálogo, home, cursos, planes, aprendizaje y ajustes.

El diseño visual mantiene la línea definida en la primera memoria. La interfaz busca ser clara, moderna y poco recargada. La **Home** se articula en hero, cursos populares, recomendaciones y categorías.



Explore actúa como catálogo filtrable.

The screenshot shows the LoopSkill 'Explore' page. At the top, there's a search bar with the text 'What do you want to learn?' and a magnifying glass icon. To the right are links for 'Explore', 'My Learning', and 'Plans', along with a user profile icon labeled 'T'. Below the search bar, the page is titled 'LOOPSKILL CATALOG' and 'Explore courses'. A subtitle reads 'Browse all categories and discover the skills you want to build next.' Below this is a row of category filters: 'All categories', 'Programming', 'Databases', 'Cloud', 'Data Science', and 'DevOps' (which is selected with a green checkmark). The main content area is titled 'DevOps' with the subtitle 'Understand automation, delivery workflows and infrastructure practices.' It displays four course cards: 'Advanced CI/CD Pipelines', 'Advanced Docker for DevOps', 'Advanced GitHub Actions', and 'Advanced Linux Administration'. Each card includes a brief description, a 'Premium' label, and a 'View course' button. Navigation arrows are visible on the left and right sides of the course cards.

Course Detail presenta información completa del curso, resultados de aprendizaje, lecciones y estado de inscripción.

The screenshot shows the 'Course Detail' page for 'Python Fundamentals'. The page has a green and white color scheme. At the top, it says 'PROGRAMMING' and 'Python Fundamentals'. A description states: 'This course introduces Python programming through practical examples and core programming concepts. You will learn variables, conditionals, loops, functions and clean problem-solving practices.' Below the description are tags for 'beginner' and 'Free', and a 'Continue course' button. The 'Course overview' section shows the instructor 'Laura Bennett', a duration of '18 hours', and '42 lessons'. The 'What you'll learn' section lists four learning objectives, each with a green checkmark: 'Write basic Python programs using variables, loops and functions.', 'Understand core programming logic and syntax.', 'Solve beginner-friendly coding exercises.', and 'Build confidence to continue into more advanced Python topics.'

Course Player añade una experiencia de aprendizaje más completa, con lista de lecciones, reproducción de vídeo y actualización del progreso.

LoopSkill

What do you want to learn?

Q

Explore

My Learning

Plans

T

LoopSkill

learn. improve. repeat.

Lesson is about to Start

0:00 / 0:05

NOW PLAYING

Python Fundamentals: Course overview and learning goals

Programming

beginner

Python Fundamentals

Instructor : Laura Bennett

Introduce the purpose of Python Fundamentals, the expected learning path and how the course is organized.

Back to course details

Course content

10 lessons

Python Fundamentals: Course overv...07:00

Setting up for Python Fundamentals08:00

Core syntax and first exercise09:00

Working with data structures10:00

Control flow and reusable logic11:00

Organizing code in modules06:00

Debugging and common mistakes07:00

Guided project in Python Fundament...08:00

Review and best practices09:00

Python Fundamentals: final review10:00

9

La página **Plans** muestra los distintos planes de suscripción disponibles en LoopSkill, permitiendo al usuario comparar sus características, identificar su plan actual y simular una mejora de suscripción según sus necesidades de aprendizaje.

PLANS & PRICING

Choose the plan that supports your learning path

Compare available plans, review what is included in each option and upgrade your access whenever you are ready to unlock more courses.

YOUR CURRENT PLAN **Free**

Current plan

Free

Start learning with access to essential courses and core platform features.

- ✓ Access to beginner courses
- ✓ Personalized recommendations
- ✓ Progress tracking
- ✓ Learning dashboard

Current plan

Most popular

Pro

\$12.99 / month

Unlock a wider catalog with intermediate content and a more complete learning experience.

- ✓ Everything in Free
- ✓ Access to intermediate courses
- ✓ Expanded course catalog
- ✓ Priority access to new content
- ✓ More advanced learning paths

Upgrade to Pro

Premium

\$24.99 / month

Get full access to the complete catalog and premium learning features.

- ✓ Everything in Pro
- ✓ Access to advanced courses
- ✓ Full catalog access
- ✓ Premium learning experience
- ✓ Priority support

Upgrade to Premium

Settings concentra la gestión de cuenta, contraseña e intereses. También muestra un panel de administración cuando el usuario tiene rol admin. Esta decisión mantiene separadas la gestión personal y el seguimiento del aprendizaje, que queda reservado a My Learning.

LoopSkill

What do you want to learn?

ExploreMy LearningPlans

T

USER SETTINGS

Manage your account

Update your personal information and choose the new skills you want to learn.

T

Test User Deployment

testdeploy@loopskill.com

CLIENT TYPE

student

CURRENT PLAN

Free

AccountPasswordInterestsLogout

Account

Update the main information associated with your LoopSkill account.

Name

Test User Deployment

Email

testdeploy@loopskill.com

Client type

student

Current plan

Free

Manage plan

Save account changes

En el backend, la estructura se organiza de forma sencilla y mantenible. Las rutas se definen en routes, la lógica de negocio en controllers, la autenticación en middleware, la conexión MySQL en config, y las respuestas y validaciones comunes en utils. Esta organización es suficiente para el alcance actual y permite seguir creciendo sin mezclar responsabilidades.

```
backend/
|-- app.js
|-- server.js
|-- package.json
|-- package-lock.json
|-- .env.example
|-- config/
|  |-- db.js
|-- controllers/
|  |-- auth.controller.js
|  |-- courses.controller.js
|  |-- enrollments.controller.js
|-- routes/
|  |-- auth.routes.js
|  |-- courses.routes.js
|  |-- enrollments.routes.js
|-- middleware/
|  |-- auth.middleware.js
|-- utils/
|  |-- db-errors.js
|  |-- http.js
|  |-- validation.js
```

4. Desarrollo del proyecto: funcionalidades y etapas

El desarrollo funcional de LoopSkill puede entenderse como una evolución desde un frontend mock hacia una aplicación integrada con backend real.

La primera etapa consistió en levantar el proyecto Angular, definir el routing, crear componentes principales y construir una identidad visual coherente. En esta fase se desarrollaron Home, Explore, Course Detail, Plans, autenticación mock y primeras estructuras de datos.

La segunda etapa introdujo servicios frontend con persistencia local. AuthService, CourseService y EnrollmentService permitieron que la interfaz se comportara como una aplicación real aunque todavía no consumiera una API. Esto permitía registrar usuarios, simular login, seleccionar intereses, inscribirse en cursos y cambiar planes.

La tercera etapa incorporó el backend Express. Se implementaron rutas de autenticación con registro, login, recuperación del usuario autenticado, validación JWT, actualización de perfil, contraseña e intereses. También se conectó el backend a MySQL/MariaDB mediante un pool de conexiones.

La cuarta etapa amplió la API de cursos. El backend permite listar cursos desde base de datos, filtrar por categoría, nivel, búsqueda y tags, obtener cursos populares, recomendaciones según intereses, detalle de curso, resultados de aprendizaje y lecciones.

La quinta etapa integró inscripciones y progreso. El usuario puede inscribirse en cursos, consultar sus cursos, separarlos entre in progress y completed, y actualizar el progreso desde Course Player. Esto supone una mejora importante respecto a la versión inicial, ya que My Learning y Course Detail ya no dependen solo de datos mock.

La sexta etapa se centró en ajustes y administración. Settings permite actualizar datos personales, cambiar contraseña y modificar intereses contra la API real. El panel admin permite crear, editar y eliminar cursos, pero esta parte sigue funcionando localmente mediante LocalStorage, no contra endpoints backend de administración.

La tabla siguiente resume el alcance real de la versión actual:

Bloque funcional	Estado actual	Observaciones
Autenticación y registro	Implementado e integrado	API Express, bcrypt, JWT y persistencia local del token
Onboarding de intereses	Implementado e integrado	Usa etiquetas de la base de datos y actualización por API
Home	Implementado	Usa cursos populares y recomendaciones desde backend
Explore	Implementado e integrado	Catálogo desde API con filtros por categoría, nivel, tags y búsqueda
Course Detail	Implementado e integrado	Incluye curso, outcomes, lessons y enrollment si hay usuario autenticado
Course Player	Implementado e integrado	Lecciones desde API y progreso actualizado en backend
Planes y suscripciones	Mock funcional	La comparación y el cambio de plan siguen en frontend/localStorage
My Learning y progreso	Implementado e integrado	Inscripciones reales por API
Settings de usuario	Implementado e integrado	Perfil, contraseña e intereses contra backend
Panel admin de cursos	Parcial	Create, Edit y Delete existen, pero siguen en LocalStorage
Base de datos	Implementada	Esquema mooc_db_v4.sql con cursos, usuarios, planes, tags, lecciones e inscripciones
Despliegue AWS	Preparado documentalmente	Guía para S3/CloudFront, Elastic Beanstalk y RDS

5. Base de datos: diseño y estructura

La base de datos responde a una estructura relacional coherente con el dominio MOOC. Las tablas principales son:

Tabla	Función
categories	Clasificación general del catálogo
plans	Planes Free, Pro y Premium
plan_features	Características ordenadas de cada plan
tags	Etiquetas tecnológicas e intereses
users	Datos del usuario, rol, tipo de cliente y plan
courses	Catálogo de cursos con categoría, nivel, plan requerido e instructor
course_outcomes	Resultados de aprendizaje de cada curso
lessons	Lecciones asociadas a cada curso
course_tags	Relación muchos a muchos entre cursos y etiquetas
user_interests	Intereses seleccionados por cada usuario
enrollments	Inscripciones y porcentaje de progreso

El modelo permite representar las necesidades principales del sistema. **users** incluye **role**, **client_type** y **plan_id**, por lo que el frontend puede distinguir entre estudiantes y administradores sin introducir campos artificiales. **courses** contiene título, descripción, categoría, nivel, plan requerido, imagen, duración, número de lecciones e instructor. **lessons** permite que Course Player funcione con contenido estructurado por curso.

Las relaciones `course_tags` y `user_interests` sostienen la lógica de recomendaciones. El backend cruza los intereses del usuario con las etiquetas de los cursos y completa resultados con cursos relacionados o populares cuando no hay suficientes coincidencias. Esta aproximación no es todavía un algoritmo avanzado, pero sí una base funcional y extensible.

`enrollments` representa la relación entre usuario y curso. El campo `progress` permite separar cursos en progreso y completados, alimentar My Learning y actualizar el avance desde el reproductor de curso.

La tabla `plan_features` normaliza las características de cada plan. Aunque la pantalla Plans del frontend sigue usando datos mock, el diseño de base de datos ya permite que esa información sea servida por la API en una fase posterior.

6. Pruebas, errores cometidos y soluciones planteadas

A lo largo del desarrollo se han realizado pruebas manuales continuas sobre rutas, formularios, persistencia local, comunicación con la API, validación de tokens y estados de interfaz. También se han ejecutado comprobaciones de compilación del frontend y verificaciones puntuales de endpoints del backend.

Uno de los principales retos fue la transición desde mocks hacia backend real. En la primera versión, varias pantallas dependían directamente de datos locales. La solución fue ir desplazando la lógica hacia servicios Angular y después sustituir progresivamente la fuente de datos por llamadas HTTP.

También se detectaron problemas asociados a datos antiguos en LocalStorage. Al convivir información mock con respuestas reales de API, algunas claves locales podían conservar estructuras anteriores. La solución consistió en normalizar la lectura del usuario actual, guardar el token en una clave separada y mantener funciones de limpieza de sesión.

En el backend fue necesario reforzar la validación de entrada. Se incorporaron validaciones para email, nombre, contraseña, tipo de cliente, intereses existentes y progreso entre 0 y 100. También se gestionan errores habituales como email

duplicado, credenciales inválidas, curso inexistente, inscripción duplicada o cuerpo JSON malformado.

La integración de CORS también requirió una decisión específica. En desarrollo se permite CORS abierto para facilitar el trabajo local con Angular en localhost:4200, mientras que en despliegue se puede restringir el origen mediante FRONTEND_ORIGIN.

La tabla siguiente recoge un resumen de incidencias relevantes:

Incidencia	Causa principal	Solución adoptada
Dependencia excesiva de mocks	Datos importados directamente en componentes	Centralización en servicios Angular
Sesión inconsistente	Usuario y token mezclados con datos locales previos	Separación de claves y carga desde /auth/me
Formularios con datos inválidos	Falta de validación robusta en backend	Utilidades de validación y respuestas 400 detalladas
Inscripciones duplicadas	Usuario podía repetir curso	Comprobación previa y manejo de error 409
Recomendaciones incompletas	Pocos cursos coincidentes con intereses	Relleno con cursos relacionados y populares
Plans no integrado	Falta de endpoints de planes	Se mantiene como mock funcional
Admin CRUD no integrado	Falta de endpoints protegidos para cursos	CRUD local mediante LocalStorage como solución temporal

En conjunto, estas incidencias han servido para reforzar dos ideas: la primera, que incluso en una fase académica conviene separar interfaz, servicios y backend; la segunda, que la integración progresiva permite avanzar sin bloquear el desarrollo visual ni la experiencia del usuario.

7. Conclusiones

El estado actual de LoopSkill representa una evolución importante respecto a la primera memoria. El proyecto ya no es solo un frontend funcional en modo mock, sino una aplicación con arquitectura cliente-servidor real y una integración significativa entre Angular, Express y MySQL/MariaDB.

A nivel funcional, el usuario puede registrarse, iniciar sesión, seleccionar intereses, explorar cursos, ver detalles completos, consultar lecciones, inscribirse, avanzar en el reproductor, actualizar progreso, consultar cursos en marcha o completados, editar su perfil, cambiar contraseña y actualizar intereses. A nivel técnico, el backend ofrece autenticación con JWT, validaciones, consultas SQL, recomendaciones basadas en intereses e inscripciones persistidas en base de datos.

El principal límite actual se encuentra en las partes que todavía permanecen en modo mock: la pantalla de planes y el panel administrativo de cursos. La base de datos ya contempla planes y características, pero falta exponer endpoints específicos para consultarlos y actualizar el plan del usuario. Del mismo modo, el panel admin permite crear, editar y eliminar cursos en local, pero todavía no existe un CRUD backend protegido para persistir esos cambios en MySQL.

Como trabajo inmediato para la siguiente versión resultan prioritarios cuatro frentes: implementar endpoints de planes y cambio de plan, sustituir el CRUD admin local por endpoints reales protegidos por rol, revisar la documentación de despliegue para reflejar todos los endpoints ya disponibles, y completar el despliegue en AWS con frontend, backend y base de datos conectados.

En definitiva, LoopSkill cubre una parte muy significativa de lo prometido en el anteproyecto y en la primera entrega. La aplicación conserva una identidad visual clara, ha ganado una API funcional y está preparada para completar su integración final de forma ordenada.

8. Bibliografía

- Amazon Web Services. (s. f.). AWS Documentation. <https://aws.amazon.com/documentation/>
- Angular. (s. f.). Angular Documentation. <https://angular.io/docs>
- Express.js. (s. f.). Express Documentation. <https://expressjs.com/>
- MySQL. (s. f.). MySQL Documentation. <https://dev.mysql.com/doc/>
- Node.js. (s. f.). Node.js Documentation. <https://nodejs.org/en/docs>